

LABORATORIO 12

¿Se puede preguntar por lo que no pasó?

ksqlDB, y una consulta que se escribe antes de que exista el dato que la contesta.

Guía para hacerlo por tu cuenta, paso a paso

Confluent Platform 8.2.0 · 3 brokers KRaft · Schema Registry · ksqlDB

Duración estimada · 40 minutos

NovaTech Logistics · caso de estudio

Antes de empezar

Qué vas a lograr

Éste es el laboratorio más corto del curso y tiene **una sola idea adentro**. Si te llevas esa idea, sobra todo lo demás.

Vas a escribir un `SELECT` sobre datos que todavía no existen, y vas a verlo contestar cuando esos datos nazcan. **Sin volver a preguntar.**

LO QUE ESTE LABORATORIO DEMUESTRA

La pregunta se hace una vez y sigue contestando.

Qué necesitas tener listo

Requisito	Cómo lo compruebas
Docker Desktop corriendo, con 6 GB de RAM	Ajustes de Docker Desktop, pestaña <i>Resources</i>
Git Bash abierto en la carpeta del lab	<code>cd Capitulo_5/lab-12-ksqldb</code>
Dos terminales , no una	El paso 4 las necesita a la vez. Ábrelas ahora, las dos en la carpeta del lab
Los puertos 9092, 9093, 9094, 8088 , 8081 y 8090 libres	El paso 1 te avisa si no lo están
Haber hecho el laboratorio 11	Hoy se usan sus contratos de Avro, y el paso 1 los registra otra vez

LO MÁS IMPORTANTE DE ESTA GUÍA ESTÁ EN EL PASO 1, Y NO ES OPCIONAL

El `bin/start-lab.sh` levanta el clúster **pero no siembra los datos**. Sin ellos, el paso 2 no tiene nada sobre lo que montarse y el paso 3 devuelve una lista vacía. **El paso 1 los siembra, y tarda alrededor de un minuto.** No te lo saltes.

El caso

El equipo de analítica de **NovaTech Logistics** pide algo que suena razonable. *Queremos saber cuántos comprobantes por minuto están entrando, y cuáles superan cierto monto.*

Y la respuesta que reciben hoy tiene siempre la misma forma. **Mañana.** Porque el camino es que los datos se copien a algún sitio durante la noche, que un proceso los agregue, y que a la mañana siguiente haya un informe.

Ese informe **siempre habla del pasado.** Es correcto, es útil, y llega tarde para todo lo que importa en el momento. Una carga anómala, un pico de rechazos, un contribuyente que empezó a hacer algo raro hace veinte minutos.

Y la alternativa que se propone siempre es peor de lo que parece. *Que escriban una aplicación que consuma el tópico y vaya calculando.* Ya viste en el laboratorio 09 lo que es un consumidor. Veinte líneas para conectarse, y todo lo demás hay que escribirlo. **El estado, las ventanas de tiempo, los reinicios, el paralelismo.** Para una pregunta. Y la siguiente pregunta es otra aplicación.

Hay una segunda mitad, más incómoda. **La gente que tiene las preguntas no escribe aplicaciones.** El analista que sabe qué hay que preguntar sabe SQL, no Java. Cada vez que la respuesta exige un desarrollador, la pregunta se pospone o no se hace.

LA METÁFORA · HOY NO ENTRA UN OBJETO, ENTRA UNA FORMA DE PREGUNTAR

Kafka sigue siendo **el restaurante**. El **mozo** es el broker, el **tipo de comanda** es el tópico, los **sectores del salón** son las particiones y el **pincho** del turno es el segmento.

Hoy hay dos formas de preguntar, y son personas distintas.

En el restaurante	En Kafka
El que baja al depósito a contar las comandas del mes y sube con un número	Una consulta de siempre , sobre una base de datos
El que se planta en el paso con una libreta y va anotando cada comanda que cruza, mientras cruza	Una consulta de ksqlDB , con <code>EMIT CHANGES</code>

El del depósito **pregunta una vez y obtiene una respuesta**. Cuando termina de contar, su número ya está viejo, porque mientras contaba entraron comandas nuevas. Si quiere saber cómo va ahora, tiene que volver a bajar.

El del paso **pregunta una vez y no deja de obtener respuestas**. No baja a ningún lado. Se queda ahí, y cada comanda que cruza es una respuesta más a la misma pregunta que hizo al principio.

Y de ahí sale lo único que asombra de ksqlDB. **El del paso puede plantarse antes de que exista la comanda**. Puede hacer la pregunta cuando todavía no hay nada que contestar, y esperar.

Las tres piezas, con sus nombres

CONCEPTO · KSQLDB

Un servicio **aparte del clúster**, aquí el contenedor `ksqldb-server`, que responde en `http://localhost:8088`. Acepta SQL y lo traduce a aplicaciones de Kafka. **No es una base de datos**. No guarda tus datos, porque los datos siguen viviendo en los tópicos. Es un traductor.

CONCEPTO · STREAM

La declaración de que un tópico se puede leer como si fuera una tabla, o sea qué columnas tiene y de qué tipo es cada una. **Declarar un stream no copia nada ni mueve nada**. Es ponerle un nombre y una forma a un tópico que ya existe, para poder escribirle SQL encima.

CONCEPTO · `EMIT CHANGES`

Las dos palabras que convierten un `SELECT` normal en uno que **no termina**. Sin ellas, ksqlDB contesta con lo que hay y se va. Con ellas, contesta con lo que hay **y se queda esperando lo que venga**.

La diferencia tiene nombres, y conviene tenerlos.

	Cómo se llama	Qué hace
Sin <code>EMIT CHANGES</code>	<i>Pull query</i>	Pregunta, responde, termina. Como cualquier SQL
Con <code>EMIT CHANGES</code>	<i>Push query</i>	Pregunta una vez, responde para siempre

TODO EL RECORRIDO VA POR `CURL`, Y HAY UNA RAZÓN HONESTA

ksqlDB trae un cliente interactivo mucho más cómodo, que está en el *para profundizar* A. **Para explorar es lo que hay que usar.**

El recorrido no lo usa porque **su salida es una tabla cuyo ancho depende del ancho de tu terminal**, así que no se puede prometer línea por línea en una guía. Lo que ves en tu pantalla no sería lo que ve otro. El `curl` **devuelve siempre lo mismo**, y por eso es el que lleva las salidas medidas de esta guía.

Levantar el clúster y sembrar los datos

Qué vamos a hacer

Encender el laboratorio y, después, **sembrar los datos a mano**. Son dos cosas distintas y la segunda no la hace el script.

Primero, levantar

```
cd Capitulo_5/lab-12-ksqldb
chmod +x bin/*.sh schema-cli/*.sh kafka-cli/*.sh ksql-cli/*.sh
bin/start-lab.sh
```

EL FINAL DE LO QUE VAS A VER · RECORTADO

```
Kafbat UI:          http://localhost:8090
Schema Registry:    http://localhost:8081    ← API REST
ksqlDB Server:      http://localhost:8088    ← API REST
ksqlDB CLI:         ksql-cli/ksql-shell.sh

Tópicos del Lab 12:
  novatech.lab10.pedidos    - 12 particiones, datos Avro
  novatech.lab10.clientes   - 3 particiones, datos Avro
```

Los dos tópicos están creados y vacíos. Fíjate en que el de pedidos tiene **12 particiones**, porque eso va a explicar algo raro en el paso 3.

Ahora, sembrar

Son cuatro comandos. Los dos primeros registran los contratos de Avro, que es el laboratorio 11 exacto. Los dos últimos escriben 30 pedidos y 5 clientes.

```
schema-cli/register-schema.sh novatech.lab10.pedidos-value infra/schemas/pedido.avsc
schema-cli/register-schema.sh novatech.lab10.clientes-value infra/schemas/cliente.avsc
kafka-cli/produce-flood-pedidos.sh 30
kafka-cli/produce-clientes-seed.sh
```

register-schema.sh — registra un contrato en el Schema Registry. **Sin contrato, ksqlDB no sabe leer los mensajes en Avro**

produce-flood-pedidos.sh 30 — escribe 30 pedidos con valores al azar. **Aquí se van casi todos los segundos**, porque cada pedido arranca una máquina virtual de Java dentro del contenedor

produce-clientes-seed.sh — escribe 5 clientes, que se usan en el paso 5

- ✓ 30 pedidos generados
- ✓ 5 clientes seed publicados

Estos IDs (1001, 1010, 1017, 1055, 1098) se usarán para los JOINS.

Medido, la siembra completa tarda 64 segundos. Es el tramo más lento del laboratorio y no hay forma de acortarlo.

VAS BIEN SI...

El tópico tiene **30 mensajes** repartidos en sus 12 particiones. Compruébalo:

```
docker exec kafka-broker-1 kafka-get-offsets \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab10.pedidos --time -1 \
  | awk -F: '{s+= $3} END {print s" mensajes en 12 particiones"}'
```

30 mensajes en 12 particiones

SI `REGISTER-SCHEMA.SH` DICE `PYTHON3: COMMAND NOT FOUND`

Es la única pieza del laboratorio que necesita Python, y **Git Bash para Windows no lo trae**. El equivalente en `curl` hace exactamente lo mismo y no depende de nada instalado. **Está medido con y sin Python, y devuelve HTTP 200 en los dos casos.**

```
curl -s -w "\nHTTP %{http_code}\n" -X POST \
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  --data '{"schema": "$(tr -d '\n' < infra/schemas/pedido.avsc | sed
  's/"\\/"/g')\n"}' \
  http://localhost:8081/subjects/novatech.lab10.pedidos-value/versions
```

Y el mismo comando cambiando `pedido.avsc` por `cliente.avsc` y `pedidos-value` por `clientes-value`. **Es el comando del laboratorio 11**, tal cual.

SI EL CONTADOR TE DA MENOS DE 30 MENSAJES

Alguno de los 30 falló, casi siempre por falta de memoria. **No hace falta empezar de cero.**

Vuelve a correr `produce-flood-pedidos.sh` con la diferencia, o simplemente sigue: el laboratorio funciona igual con 20 pedidos que con 30.

SI EL `CURL` AL 8088 NO RESPONDE TODAVÍA

ksqlDB es lo último que arranca y tarda más que los brokers. Compruébalo antes de seguir:

```
curl -s http://localhost:8088/info
```


Ponerle forma de tabla a un tópico

Qué vamos a hacer

El tópico ya tiene 30 pedidos escritos en Avro. Kafka sabe que son bytes y el Schema Registry sabe qué campos tienen. **Lo que falta es decirle a ksqlDB que los mire como filas.**

El comando

```
curl -s -X POST http://localhost:8088/ksql \
  -H "Content-Type: application/vnd.ksql.v1+json" \
  -d '{"ksql":"CREATE STREAM pedidos_stream (id INT, cliente_id INT, producto VARCHAR,
cantidad INT, monto DOUBLE, estado VARCHAR) WITH
(KAFKA_TOPIC='\"\"\"novatech.lab10.pedidos\"\"\", VALUE_FORMAT='\"\"\"AVRO\"\"');"}' \
  | tr ',' '\n' | grep -E '"status"|"message"'
```

POST /ksql — el punto de entrada para las sentencias que **definen** cosas, o sea crear, borrar y listar. **Las consultas van por otro**, que verás en el paso 3

-H "...vnd.ksql.v1+json" — el tipo de contenido propio de ksqlDB. **Sin esta cabecera contesta 415**

-d '{"ksql": "..."}' — el SQL viaja como texto dentro de un JSON, igual que el schema del laboratorio 11

'\"\"\"' — **eso no es un error tipográfico**. Es cómo se pone una comilla simple dentro de una cadena que ya va entre comillas simples. Cierra la comilla del shell, pone una literal, y vuelve a abrirla. **Cópialo tal cual**

CREATE STREAM ... (columnas) — los seis campos del pedido, con su tipo. **Tienen que coincidir con lo que hay en el tópico**

KAFKA_TOPIC= — **sobre qué tópico existente**. El stream no crea un tópico, se monta sobre uno que ya está

VALUE_FORMAT='AVRO' — cómo están escritos los mensajes. ksqlDB irá al Schema Registry a buscar el schema

| tr ',' '\n' | grep -E ... — la respuesta es un JSON de 447 caracteres en una sola línea, con la sentencia entera repetida dentro. **Esto la parte por comas y deja solo las dos líneas que importan**. Es un corte bruto y no un formateador de JSON

LO QUE VAS A VER

```
"commandStatus":{"status":"SUCCESS"}
"message":"Stream created"
```

Cómo se lee

`SUCCESS` y `Stream created`. Y lo importante es lo que no pasó. No se copió un solo mensaje, no se creó un tópico nuevo, no se movió un byte.

Lo único que hay ahora es **una declaración**. ksqlDB sabe que ese tópico se puede leer como una tabla de seis columnas.

VAS BIEN SI...

Salieron esas dos líneas. Y puedes verlo listado:

```
curl -s -X POST http://localhost:8088/ksql \  
-H "Content-Type: application/vnd.ksql.v1+json" \  
-d '{"ksql":"SHOW STREAMS;"}' | tr ',' '\n' | grep -E '"name"|"topic"'
```

```
"name":"PEDIDOS_STREAM"  
"topic":"novatech.lab10.pedidos"  
"name":"KSQL_PROCESSING_LOG"  
"topic":"novatech_ksql_processing_log"
```

El nombre salió en mayúsculas, aunque tú lo escribiste en minúsculas. ksqlDB los convierte y eso es normal. El segundo stream lo crea ksqlDB para sí mismo.

SI DICE A STREAM WITH THE SAME NAME ALREADY EXISTS

La respuesta completa es ésta, y no es un problema:

```
"error_code":40001  
"message":"Cannot add stream 'PEDIDOS_STREAM': A stream with the same name already  
exists"
```

El stream quedó de una corrida anterior. **Sigue al paso 3, porque el que necesitas ya está.** Si prefieres empezar de cero, bórralo primero por el mismo punto de entrada, cambiando el `CREATE STREAM ...` por `DROP STREAM pedidos_stream;`.

SI CONTESTA HTTP 415 O EL GREP NO ENCUENTRA NADA

Falta la cabecera `Content-Type: application/vnd.ksql.v1+json`. **Para ver la respuesta cruda y entender qué pasó**, quita el filtro del final y corre solo el `curl`.

SI DICE ALGO SOBRE EL SCHEMA O EL `VALUE_FORMAT`

El contrato de Avro no está registrado. **Vuelve al paso 1** y comprueba:

```
curl -s http://localhost:8081/subjects
```

Preguntar por lo que ya pasó

Qué vamos a hacer

Ahora el `SELECT`. Éste primero es **el aburrido a propósito**, porque pregunta por lo que ya está escrito, que es lo que haría cualquier base de datos.

Sirve para comprobar dos cosas antes del paso que importa. Que el SQL funciona, y que los datos se leen bien.

El comando

```
curl -s -X POST http://localhost:8088/query-stream \
  -H "Content-Type: application/vnd.ksqlapi.delimited.v1" \
  -d '{"sql":"SELECT id, producto, monto FROM pedidos_stream EMIT CHANGES LIMIT 3;","properties":{"auto.offset.reset":"earliest"}}'
```

POST /query-stream — otro punto de entrada. Las consultas no van por `/ksql`. Van por aquí, que es el que sabe devolver una respuesta que llega de a poco

-H "...ksqlapi.delimited.v1" — pide la respuesta **delimitada por líneas**, o sea una línea por fila, a medida que salen

EMIT CHANGES — *push query*, así que no termina sola

LIMIT 3 — lo único que hace que termine. Sin el `LIMIT`, este comando se queda corriendo hasta que lo cortes con `Ctrl+C`

"properties":{"auto.offset.reset":"earliest"} — empieza por el principio del tópico, porque queremos ver lo viejo. **Es el mismo parámetro del laboratorio 09**

LO QUE VAS A VER

```
{ "queryId": "transient_PEDIDOS_STREAM_5830900987375917743", "columnNames":
  [ "ID", "PRODUCTO", "MONTO" ], "columnTypes": [ "INTEGER", "STRING", "DOUBLE" ] }
[ 24, "Cinta industrial", 8829.0 ]
[ 7, "Cartón premium", 6695.0 ]
[ 15, "Cuerda nautica", 31554.0 ]
```

Línea	Qué dice
La primera	La cabecera . El identificador de la consulta y, sobre todo, los nombres y tipos de las columnas
Las tres siguientes	Una fila cada una, como lista JSON, en el orden de las columnas de la cabecera

Los productos y montos que te salgan van a ser otros, porque los 30 pedidos se generaron con valores al azar.

Y ahora lo importante de este paso, que hay que medir para crearlo

Corre **el mismo comando cinco veces seguidas**, sin tocar nada entremedio, y mira solo los `id`.

LO QUE SALIÓ · MEDIDO, CINCO CORRIDAS

```

corrida 1:  15  10  12
corrida 2:   5   7  10
corrida 3:   5  14  19
corrida 4:   5  12  18
corrida 5:  15  12  13

```

El mismo `SELECT`, sobre datos que no cambiaron, devuelve filas distintas cada vez. En una base de datos eso sería un fallo grave. Aquí es el comportamiento correcto.

CONCEPTO · PARTICIÓN, OTRA VEZ

Cada uno de los pedazos en que se corta un tópico. Éste tiene **12**. El orden de los mensajes está garantizado **dentro de cada partición**, y en ninguna parte entre ellas.

ksqlDB lee las 12 particiones a la vez, y el `LIMIT 3` se queda con **las tres primeras que lleguen**, que dependen de cuál contestó antes.

Y ESTO HAY QUE DECIRLO ANTES DE QUE ALGUIEN ESCRIBA UN INFORME

Un flujo no tiene un orden global, así que no hay `SELECT` que pueda dártelo. Cualquier informe que dependa del orden entre particiones está mal planteado. Éste es el momento de decirlo, no cuando ya esté escrito.

VAS BIEN SI...

Corriste el comando cinco veces y **al menos dos corridas te dieron listas distintas**. Si las cinco te dieron lo mismo, no está mal, pero corre unas cuantas más. **Anota una respuesta**. ¿Por qué esto no es un fallo?

SI EL COMANDO SE QUEDA CORRIENDO Y NO VUELVE

Se te olvidó el `LIMIT 3`. Un `EMIT CHANGES` sin `LIMIT` **no termina nunca**, y eso es exactamente lo que el paso 4 va a aprovechar. Sal con `Ctrl+C`.

SI SOLO SALE LA CABECERA Y NINGUNA FILA

O el tópico está vacío, o se te olvidó el `"auto.offset.reset":"earliest"`. **Sin él, ksqlDB empieza por el final** y no hay nada nuevo que traer. Comprueba el contador del paso 1.

Preguntar por lo que todavía no pasó

Qué vamos a hacer

Éste es el laboratorio. Todo lo anterior era preparación.

Vamos a lanzar **la misma consulta**, cambiando una sola cosa. `earliest` por `latest`. Con eso la consulta ignora los 30 pedidos que ya están y **se queda esperando lo que venga**.

PREDICE ANTES DE EJECUTAR, Y ESCRÍBELO

Con `latest` y el tópico quieto, ¿qué va a imprimir la consulta? **¿Va a dar error, va a terminar, o se va a quedar ahí?** Contesta antes de teclear. El paso vale la mitad si lees la respuesta antes de haber apostado.

En la terminal A

```
curl -s -N -X POST http://localhost:8088/query-stream \
  -H "Content-Type: application/vnd.ksqlapi.delimited.v1" \
  -d '{"sql":"SELECT id, producto, monto FROM pedidos_stream EMIT CHANGES LIMIT
1;","properties":{"auto.offset.reset":"latest"}}'
```

-N — sin memoria intermedia. Le dice a `curl` que imprima cada línea en cuanto llegue, en vez de juntarlas. **Sin esto no verías nada hasta el final, y el laboratorio entero se pierde**

"auto.offset.reset":"latest" — el cambio. Solo lo que llegue de ahora en adelante

LIMIT 1 — con un solo mensaje ya está demostrado. Sin el `LIMIT`, se sale con `Ctrl+C`

LO QUE VAS A VER, AL INSTANTE

```
{"queryId":"transient_PEDIDOS_STREAM_527849346038229575","columnNames":
["ID","PRODUCTO","MONTO"],"columnTypes":["INTEGER","STRING","DOUBLE"]}
```

Y después, nada. La cabecera y el cursor parpadeando.

ESA NADA ES EL RESULTADO

La consulta **no falló, no terminó y no está colgada. Está esperando.** Ya sabe qué columnas va a devolver, y lo dice la cabecera. Pero todavía no hay ninguna fila que devolver, **porque el dato no existe.**

Déjala ahí. No la cortes.

En la terminal B

```
kafka-cli/produce-pedido-avro.sh 777 1001 "Pedido en vivo" 1 99999.99 pendiente
```

`produce-pedido-avro.sh` — envoltorio del curso. Escribe **un** pedido en Avro en el tópico. **No sabe nada de ksqldb** ni de que hay una consulta esperando

`777 ... pendiente` — los seis campos del pedido. **Elige un texto reconocible** para el producto, porque lo vas a ver aparecer en la otra terminal

SALE UN AVISO QUE NO ES TUYO

El envoltorio imprime `Warning: --property is deprecated and will be removed in a future version`. **Sale siempre y no afecta a nada.** Es del script, no de lo que tú escribiste.

Vuelve a mirar la terminal A

ÉSTA ES LA CORRIDA MEDIDA, CON LA HORA REAL DE CADA LÍNEA

```
[00:27:49] {"queryId":"transient_PEDIDOS_STREAM_527849346038229575","columnNames":
["ID","PRODUCTO","MONTO"],...}

... diez segundos sin nada ...

[00:27:59] <- aquí se produjo el pedido en la terminal B
[00:28:01] [777,"Pedido en vivo",99999.99]
```

Cómo se lee

Aquí está el laboratorio entero, en tres marcas de tiempo.

Hora	Qué pasó
00:27:49	La pregunta se hizo. En este momento el dato no existía en ninguna parte
00:27:59	Diez segundos después, la consulta seguía viva y sin nada que decir. Y en ese instante nació el dato
00:28:01	Dos segundos después de que el dato naciera, la consulta lo contestó

NADIE VOLVIÓ A PREGUNTAR

El `SELECT` se escribió **una sola vez**, diez segundos antes de que existiera la fila que lo contestó. **Eso es lo que ninguna base de datos puede hacer**, y es la única razón por la que ksqlDB existe.

LO LOGRASTE SI...

En la terminal A apareció tu `"Pedido en vivo"` sin que tú tocaras esa terminal.

Compara con tu predicción. ¿Dijiste que iba a dar error, que iba a terminar, o que se iba a quedar esperando?

SI EN LA TERMINAL A NO APARECE NADA AUNQUE PRODUJISTE

Comprueba tres cosas, en este orden.

Una. Que pusiste el `-N`. Sin él, `curl` junta la salida y no verías la fila hasta el final.

Dos. Que la terminal B está en la carpeta del laboratorio. Si no, el script no existe y no produjo nada.

Tres. Que el pedido llegó de verdad. En una tercera terminal:

```
docker exec kafka-broker-1 kafka-get-offsets \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab10.pedidos --time -1 \
  | awk -F: '{s+=$3} END {print s}'
```

Tiene que decir **31**, uno más que en el paso 1.

SI LA CONSULTA IMPRIME FILAS VIEJAS EN CUANTO ARRANCA

Dejaste `earliest` en vez de `latest`. **Es el único cambio entre el paso 3 y el paso 4**, y es el que hace la demostración. Córdala y vuelve a lanzarla.

STREAM contra TABLE, y la trampa

Qué vamos a hacer

Un **STREAM** son **eventos**, o sea todo lo que pasó, y nada se pisa. Una **TABLE** es **estado**, o sea el último valor de cada clave.

Vamos a declarar una **TABLE** y a comprobar que **no se comporta como uno espera**.

Declarar la tabla

```
curl -s -X POST http://localhost:8088/ksql \
  -H "Content-Type: application/vnd.ksql.v1+json" \
  -d '{"ksql":"CREATE TABLE clientes_table (id INT PRIMARY KEY, nombre VARCHAR, tipo VARCHAR, ciudad VARCHAR) WITH (KAFKA_TOPIC='\"novatech.lab10.clientes\"', VALUE_FORMAT='\"AVRO\"', KEY_FORMAT='\"AVRO\"');"}' \
  | tr ',' '\n' | grep -E '"status"|"message"'
```

id INT PRIMARY KEY — una **TABLE** exige una clave primaria y un **STREAM** no. Es la diferencia de fondo, porque «el último valor de cada clave» no significa nada sin clave

KEY_FORMAT='AVRO' — la clave también tiene su contrato, y es el **-key** que viste aparecer en el laboratorio 11

LO QUE VAS A VER

```
"commandStatus":{"status":"SUCCESS"
"message":"Table created"
```

Produce dos veces el mismo cliente

```
kafka-cli/produce-cliente-avro.sh 1001 "Acme S.A." VIP Santiago
kafka-cli/produce-cliente-avro.sh 1001 "Acme S.A. - actualizado" VIP Santiago
```

El mismo **id**, 1001, con dos nombres distintos.

PREDICE OTRA VEZ

Si una **TABLE** guarda «el último valor de cada clave», y consultas por el **id** 1001, **¿cuántas filas esperas?**

Consulta la tabla

```
curl -s -X POST http://localhost:8088/query-stream \
  -H "Content-Type: application/vnd.ksqlapi.delimited.v1" \
  -d '{"sql":"SELECT id, nombre FROM clientes_table WHERE id = 1001 EMIT CHANGES LIMIT 3;","properties":{"auto.offset.reset":"earliest"}}'
```

LO QUE VAS A VER

```
{"queryId":"transient_CLIENTES_TABLE_952618109544783955","columnNames":
["ID","NOMBRE"],"columnTypes":["INTEGER","STRING"]}
[1001,"NovaCorp"]
[1001,"Acme S.A."]
[1001,"Acme S.A. - actualizado"]
```

Cómo se lee

Tres filas, no una. Y el primer nombre, `NovaCorp`, ni siquiera lo escribiste tú, porque venía del *seed* del paso 1.

POR QUÉ LA TABLA EMITIÓ TRES CAMBIOS

Porque le pediste una *push query*, con `EMIT CHANGES`. **Y lo que emite una *push query* sobre una tabla es cada cambio de esa tabla**, no su estado final.

La tabla **sí** tiene un solo valor para el 1001, que es el último. Lo que pasa es que le preguntaste por la película y no por la foto. **Para la foto hay que hacer una *pull query***, o sea el mismo `SELECT` sin `EMIT CHANGES`, y para eso la tabla tiene que ser materializada.

LO LOGRASTE SI...

Te salieron tres filas donde esperabas una, y puedes explicar por qué.

Anota una respuesta. ¿Qué le pedirías a ksqlDB si lo que quieres es «el nombre actual del cliente 1001» y nada más?

SI DICE ALGO SOBRE LA CLAVE O EL `KEY_FORMAT`

Falta el contrato de la clave. El *seed* de clientes escribe con clave, y **la clave tiene su propio subject, el `-key`**. Compruébalo:

```
curl -s http://localhost:8081/subjects
```

Tiene que aparecer `novatech.lab10.clientes-key`. Si no está, vuelve a correr `produce-clientes-seed.sh`.

Lo que aprendiste

1 La pregunta se hace una vez y sigue contestando.

Un `SELECT` con `EMIT CHANGES` no termina. Lo escribiste diez segundos antes de que existiera la fila que lo contestó, y nadie volvió a preguntar. Ésa es la única razón por la que ksqlDB existe.

2 ksqlDB no es una base de datos y no guarda tus datos.

Declarar un `STREAM` no copia nada ni mueve un byte. Es ponerle nombre y forma a un tópico que ya existe. Los datos siguen viviendo donde estaban, y si borras el stream, siguen ahí.

3 Un flujo no tiene orden global, y ningún `SELECT` te lo puede dar.

El mismo `SELECT` sobre datos que no cambiaron devolvió filas distintas cinco veces seguidas. El orden está garantizado dentro de cada partición y en ninguna parte entre ellas. Un informe que dependa de ese orden está mal planteado.

4 Una `TABLE` con `EMIT CHANGES` te da la película, no la foto.

Pediste el cliente 1001 y salieron tres filas, que son sus tres versiones. La tabla sí tiene un solo valor vigente. Lo que cambia es qué le preguntaste.

Para profundizar

A · El cliente interactivo

ksqlDB trae un cliente de línea de comandos con su propio *prompt*.

```
ksql-cli/ksql-shell.sh
```

Y dentro, sin `curl` ni JSON de por medio:

```
SET 'auto.offset.reset'='earliest';  
SHOW STREAMS;  
SELECT id, producto, monto FROM pedidos_stream EMIT CHANGES;
```

`Ctrl+C` corta la consulta y devuelve el *prompt*. `exit` sale.

Es mucho más cómodo que el `curl` del recorrido, y para explorar es lo que hay que usar. Compara las dos formas de hacer lo mismo, porque en producción vas a usar las dos.

La pregunta que vale. Si el *prompt* es más cómodo, ¿por qué existe la API REST? *Pista, ¿quién hace las consultas cuando no hay una persona delante?*

B · Un `SELECT` que no termina, de verdad

Todo el recorrido usó `LIMIT` para que los comandos volvieran. Quítalo y mira qué es de verdad una *push query*.

```
curl -s -N -X POST http://localhost:8088/query-stream \  
  -H "Content-Type: application/vnd.ksqlapi.delimited.v1" \  
  -d '{"sql":"SELECT id, producto, monto FROM pedidos_stream WHERE monto > 20000 EMIT CHANGES;","properties":{"auto.offset.reset":"latest"}}'
```

Y en la otra terminal, produce varios pedidos, unos por encima de 20000 y otros por debajo.

Lo que hay que mirar. Solo aparecen los que cumplen el `WHERE`. **El filtro se aplica a medida que los datos pasan**, no sobre un conjunto guardado. Sal con `Ctrl+C`, que es la forma normal de terminar una consulta de éstas.

C · Las consultas que quedan corriendo

Las del recorrido son *transient*, o sea que mueren cuando cierras el `curl`. Se puede crear una que siga corriendo en el servidor.

```
curl -s -X POST http://localhost:8088/ksql \
  -H "Content-Type: application/vnd.ksql.v1+json" \
  -d '{"ksql":"CREATE STREAM pedidos_grandes AS SELECT * FROM pedidos_stream WHERE monto > 20000 EMIT CHANGES;"}' \
  | tr ',' '\n' | grep -E '"status"|"message"'

curl -s -X POST http://localhost:8088/ksql \
  -H "Content-Type: application/vnd.ksql.v1+json" \
  -d '{"ksql":"SHOW QUERIES;"}' | tr ',' '\n' | grep -E '"id"|"state"'
```

LO QUE SALE · MEDIDO

```
"commandStatus":{"status":"SUCCESS"
"message":"Created query with ID CSAS_PEDIDOS_GRANDES_5"

"id":"CSAS_PEDIDOS_GRANDES_5"
"state":"RUNNING"
```

Lo que hay que mirar. Un `CREATE STREAM ... AS SELECT` **sí crea un tópico nuevo**, al revés que el `CREATE STREAM` del paso 2. Antes de este comando el listado solo tenía `novatech.lab10.pedidos`, y después aparece también `PEDIDOS_GRANDES`, en mayúsculas. Compruébalo:

```
docker exec kafka-broker-1 kafka-topics \
  --bootstrap-server kafka-broker-1:29092 --list | grep -i grandes
```

Y la consulta queda `RUNNING` en el servidor aunque cierres tu terminal, que es lo que hace falta para un panel o una alerta.

La pregunta que vale. ¿Quién consume ese tópico nuevo, y qué pasa si nadie lo hace?

D · Agregaciones con ventana de tiempo

Es la pregunta original del caso, o sea cuántos comprobantes por minuto.

```
curl -s -N -X POST http://localhost:8088/query-stream \
  -H "Content-Type: application/vnd.ksqlapi.delimited.v1" \
  -d '{"sql":"SELECT estado, COUNT(*) FROM pedidos_stream WINDOW TUMBLING (SIZE 1 MINUTE) GROUP BY estado EMIT CHANGES;","properties":{"auto.offset.reset":"earliest"}}'
```

LO QUE SALE · MEDIDO, CON `LIMIT 4` PARA QUE TERMINE

```
{ "queryId": "transient_PEDIDOS_STREAM_404246666237798640", "columnNames":  
  [ "ESTADO", "KSQL_COL_0" ], "columnTypes": [ "STRING", "BIGINT" ] }  
[ "enviado", 1 ]  
[ "enviado", 2 ]  
[ "enviado", 1 ]  
[ "enviado", 3 ]
```

Lo que hay que mirar. `WINDOW TUMBLING` corta el tiempo en tramos que no se solapan, y cuenta dentro de cada uno. **Es lo que el informe nocturno del caso hacía a las tres de la mañana,** hecho mientras los datos entran.

Y fíjate en que la cuenta **sube y vuelve a bajar**, o sea 1, 2, 1, 3. No es un error. **Cada vez que empieza un tramo nuevo, la cuenta arranca de cero otra vez**, y lo que estás viendo son tramos distintos entremezclados. La columna se llama `KSQL_COL_0` porque no le pusiste nombre al `COUNT (*)`. Con `COUNT(*) AS total` se llamaría `TOTAL`.

E · El reporte del laboratorio

`plantillas/reporte-entregable.md` recorre las actividades con sus preguntas. Las respuestas de referencia están en `soluciones/reporte-resuelto.md` y en `soluciones/respuestas-desafio.md`.

Antes de cerrar

DEJA EL TERRENO LIMPIO

```
bin/90-test-lab.sh    # ver el estado del laboratorio  
bin/stop-lab.sh      # apagar los contenedores
```

`bin/stop-lab.sh` solo detiene los contenedores y no borra nada. **Ojo al reanudar**, porque `bin/start-lab.sh` reconstruye el laboratorio desde cero. **Los streams que declaraste y los 30 pedidos no sobreviven**, así que al volver hay que repetir la siembra del paso 1.

LO QUE DICE EL VALIDADOR SI TODO ESTÁ EN ORDEN

```
Validador del Lab 12 – estado actual  
✓ los 3 brokers están healthy  
✓ ksqlDB server responde en :8088  
✓ SHOW STREAMS responde  
  
✓ LAB EN BUEN ESTADO (3 verificaciones OK)
```


LO QUE TE LLEVAS

El informe que llega mañana por la mañana no está mal hecho. **Está hecho de la única forma que se podía cuando la pregunta había que bajarla al depósito.**

Hoy viste la otra forma, y cabe en una línea de SQL que un analista puede escribir sin pedirle nada a un desarrollador. **Escríbela una vez, y sigue contestando.**